

Databricks Mini Project 1 Document:

The screenshot displays the Databricks Jobs & Pipelines interface. The main panel shows a job run graph with two tasks: 'Run_Mini_Project' and 'Refresh_Dashboard', both of which have succeeded. The 'Run_Mini_Project' task is a serverless job that ran for 1m 46s. The 'Refresh_Dashboard' task is a notebook job that ran for 51s. The right sidebar provides detailed information about the job run, including its ID, run ID, launch method (Manual), start and end times, duration (2m 38s), status (Succeeded), and performance optimization status (Enabled). The 'Metrics' section indicates that no query metrics are available for this job run. The 'Compute' section shows that the cluster is terminating and is a serverless job. The 'Environments' section shows that the job is running in a Notebook Environment with version 5.

Jobs & Pipelines > Mini Project 1 Job >

Mini Project 1 Job run Improved Performance Observability: ON [Send feedback](#)

Graph Timeline List

Run_Mini_Project Succeeded · 1m 46s
...@gmail.com/superstore_mini_project
Serverless

Refresh_Dashboard Succeeded · 51s
Mini Project 1 Dashboard

Job run details

Job ID: 157605558928179
Job run ID: 626093139526693
Launched: Manually
Started: Sep 12, 2026, 05:14 PM
Ended: Sep 12, 2026, 05:17 PM
Duration: 2m 38s
Status: Succeeded
Lineage: No lineage information for this job. [Learn more](#)
Performance optimized: Enabled
[View run events](#)

Metrics

Query metrics: No query metrics available for this job run.

Compute

Cluster is terminating
Serverless
Usage tags: [Query history](#) [Logs](#)

Environments

Notebook Environment
Environment version: 5

Successful in getting full pipeline running

The screenshot shows the Databricks 'store' directory view. The 'Overview' tab is selected, displaying the path '/Volumes/mini_projects/mp1/store'. A search bar is present with the text 'Filter files and directories at this level'. Below the search bar, a table lists the files in the directory. The table has three columns: 'Name', 'Size', and 'Last modified'. A single file, 'superstore.csv', is listed with a size of 2.23 MB and was last modified 1 hour ago.

store

Overview Files Details Permissions Policies

Description

[AI generate](#) [Add](#)

/Volumes/mini_projects/mp1/store [Refresh](#) [Create directory](#)

Filter files and directories at this level

Name	Size	Last modified
superstore.csv	2.23 MB	1 hour ago

superstore.csv stored in a mini_projects volume with mp1 schema

```
▶ ✓ 04:47 PM (<1s) 1
from pyspark.sql import functions as F

▶ ✓ 04:47 PM (2s) 2 Python
file_path = "/Volumes/mini_projects/mpl/store/superstore.csv"

raw_df = (
    spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .option("quote", "'")
    .option("escape", "'")
    .csv(file_path)
)

#Done for cleaner headings
clean_cols = [c.strip().replace(" ", "_") for c in raw_df.columns]
df = raw_df.toDF(*clean_cols)

df.printSchema()
display(df.limit(5))

> See performance \(1\) ✓ Checked for improvements
> 2 dataframes: df, raw_df
```

```
▶ ✓ 04:47 PM (2s) 3

# 1. How many total number of customers we have ?

total_customers = df.select("Customer_id").distinct().count()
print(f"Total customers: {total_customers}")

spark.createDataFrame([(total_customers,)], ["total_customers"]) \
    .write.mode("overwrite").saveAsTable("gold_total_customers")

> See performance \(2\) ✓ Checked for improvements
Total customers: 793
```

```
▶ ✓ 04:47 PM (3s) 4

# 2. total number of orders we have

total_orders = df.select("Order_id").distinct().count()
print(f"Total orders: {total_orders}")

spark.createDataFrame([(total_orders,)], ["total_orders"]) \
    .write.mode("overwrite").saveAsTable("gold_total_orders")

> See performance \(2\) ✓ Checked for improvements
Total orders: 5009
```

```
▶ ✓ 04:47 PM (4s) 5

# 3. Total number of sales as of now

total_sales = df.agg(F.sum("Sales").alias("total_sales")).collect()[0]["total_sales"]
print(f"Total sales: {total_sales:,.2f}")

spark.createDataFrame([(float(total_sales),)], ["total_sales"]) \
    .write.mode("overwrite").saveAsTable("gold_total_sales")

> See performance \(2\) ✓ Checked for improvements
Total sales: 2,297,200.86
```

04:47 PM (3s) 6

```
# 4. Total profit

total_profit = df.agg(F.sum("Profit").alias("total_profit")).collect()[0]["total_profit"]
print(f"Total profit: {total_profit:,.2f}")

spark.createDataFrame([(float(total_profit),)], ["total_profit"]) \
    .write.mode("overwrite").saveAsTable("gold_total_profit")
```

> [See performance \(2\)](#) ✓ Checked for improvements

Total profit: 286,397.02

04:47 PM (4s) 7

```
# 5. Top sales by country

top_sales_by_country = (
    df.groupBy("Country")
      .agg(F.sum("Sales").alias("total_sales"))
      .orderBy(F.desc("total_sales"))
)

display(top_sales_by_country)

top_sales_by_country.write.mode("overwrite").saveAsTable("gold_sales_by_country")
```

> [See performance \(2\)](#) ✓ Checked for improvements

top_sales_by_country: pyspark.sql.connect.dataframe.DataFrame = [Country: string, total_sales: double]

04:47 PM (7s) 8

```
# 6. Most profitable region and country

most_profitable_region = (
    df.groupBy("Region")
      .agg(F.sum("Profit").alias("total_profit"))
      .orderBy(F.desc("total_profit"))
)

most_profitable_country = (
    df.groupBy("Country")
      .agg(F.sum("Profit").alias("total_profit"))
      .orderBy(F.desc("total_profit"))
)

display(most_profitable_region)
display(most_profitable_country)

most_profitable_region.write.mode("overwrite").saveAsTable("gold_profit_by_region")
most_profitable_country.write.mode("overwrite").saveAsTable("gold_profit_by_country")
```

> [See performance \(4\)](#) ✓ Checked for improvements

> 2 dataframes: most_profitable_country, most_profitable_region

04:47 PM (4s)9

Python

7. Top sales category products

```
top_sales_category = (
    df.groupby("category")
    .agg(F.sum("sales").alias("total_sales"))
    .orderBy(F.desc("total_sales"))
)

display(top_sales_category)

top_sales_category.write.mode("overwrite").saveAsTable("gold_sales_by_category")
```

> [See performance \(2\)](#) ✓ Checked for improvements

> top_sales_category: pyspark.sql.connect.dataframe.DataFrame = [Category: string, total_sales: double]

Table

	Category	total_sales
1	Technology	836154.0329999966
2	Furniture	741999.7952999998
3	Office Supplies	719047.0320000029

3 rows

4.286s runtime

Refreshed 40m ago

04:47 PM (4s)10

Python

8. Top 10 sales sub category products

```
top10_sub_category = (
    df.groupby("sub_category")
    .agg(F.sum("sales").alias("total_sales"))
    .orderBy(F.desc("total_sales"))
    .limit(10)
)

display(top10_sub_category)

top10_sub_category.write.mode("overwrite").saveAsTable("gold_top10_sub_category")
```

> [See performance \(2\)](#) ✓ Checked for improvements

> top10_sub_category: pyspark.sql.connect.dataframe.DataFrame = [Sub_Category: string, total_sales: double]

04:47 PM (4s)11

Python

9. Most ordered quantity product

```
most_ordered_product = (
    df.groupby("Product_Name")
    .agg(F.sum("quantity").alias("total_quantity"))
    .orderBy(F.desc("total_quantity"))
)

display(most_ordered_product.limit(10))

most_ordered_product.write.mode("overwrite").saveAsTable("gold_most_ordered_product")
```

> [See performance \(2\)](#) ✓ Checked for improvements

> most_ordered_product: pyspark.sql.connect.dataframe.DataFrame = [Product_Name: string, total_quantity: long]

Table

	Product_Name	total_quantity
1	Staples	215
2	Staple envelope	170
3	Easy-staple paper	150
4	Staples in misc. colors	86
5	KI Adjustable-Height Table	74
6	Avery Non-Stick Binders	71
7	Storex Dura Pro Binders	71
8	GBC Premium Transparent Covers with Diagonal Lined Pattern	67
9	Situations Contoured Folding Chairs, 4/Set	64
10	Staple-based wall hangings	62

10 rows

4.162s runtime

Refreshed 40m ago

```
# 10. Top customer based on sales , city

top_customers_by_sales_city = (
    df.groupby("customer_name", "city")
    .agg(F.sum("Sales").alias("total_sales"))
    .orderBy(F.desc("total_sales"))
)

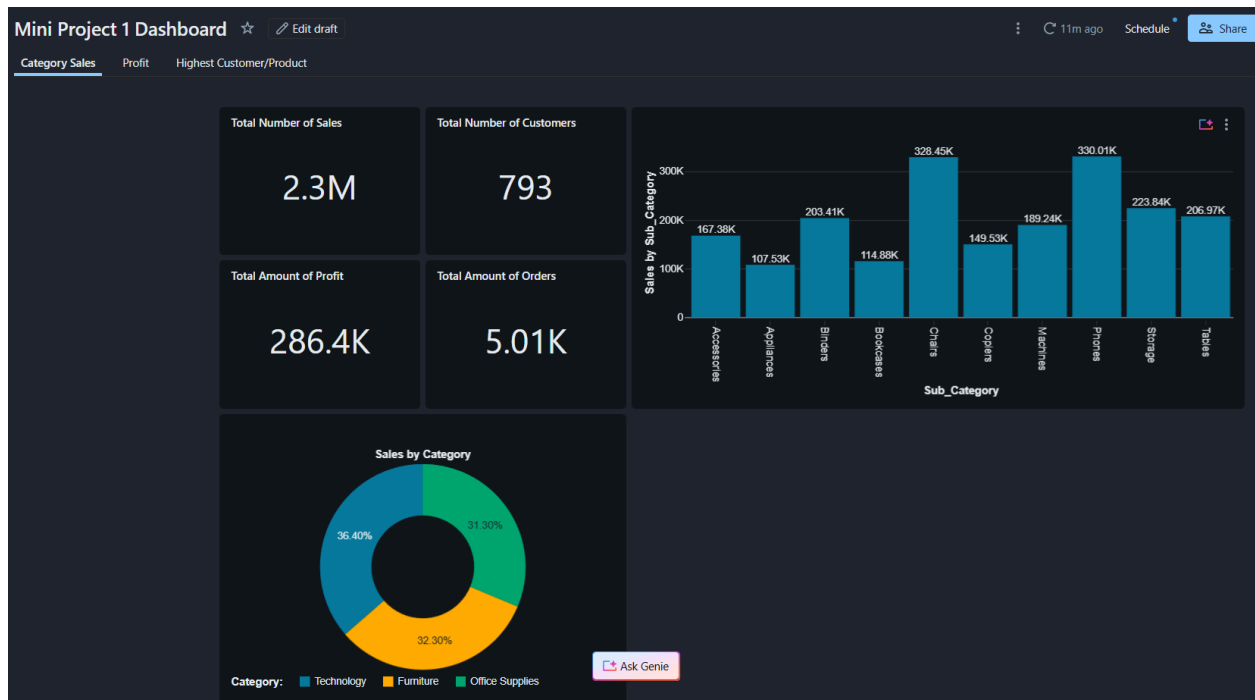
display(top_customers_by_sales_city.limit(10))

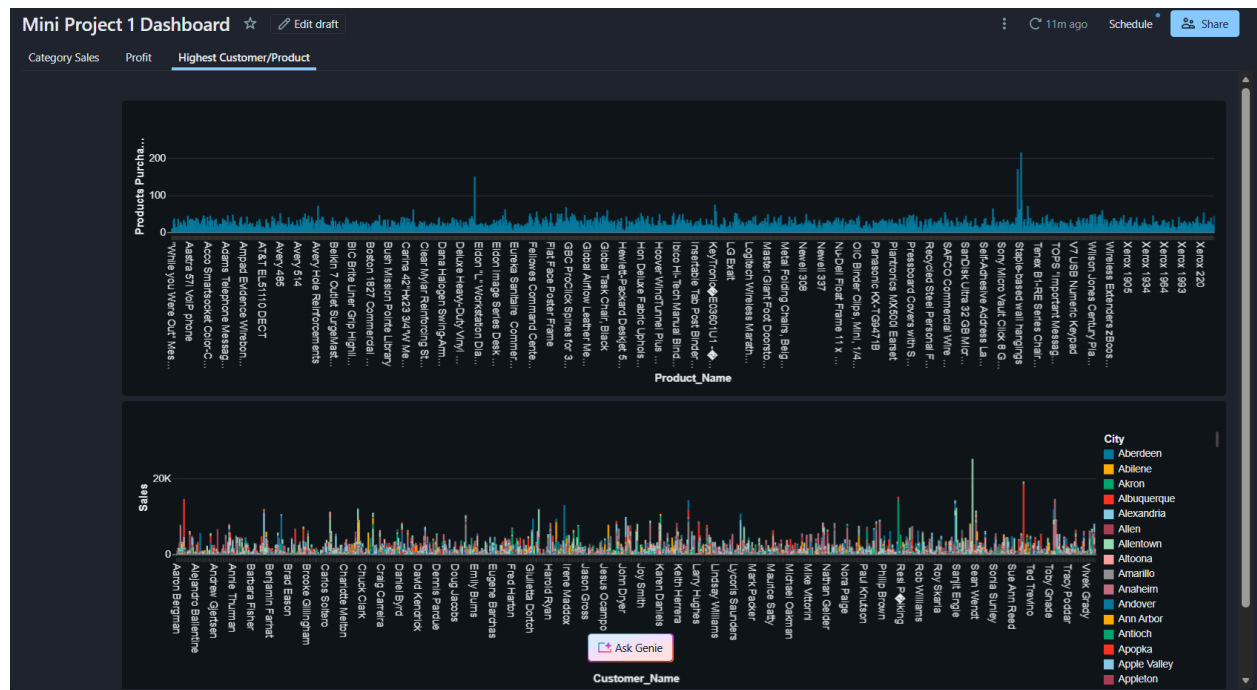
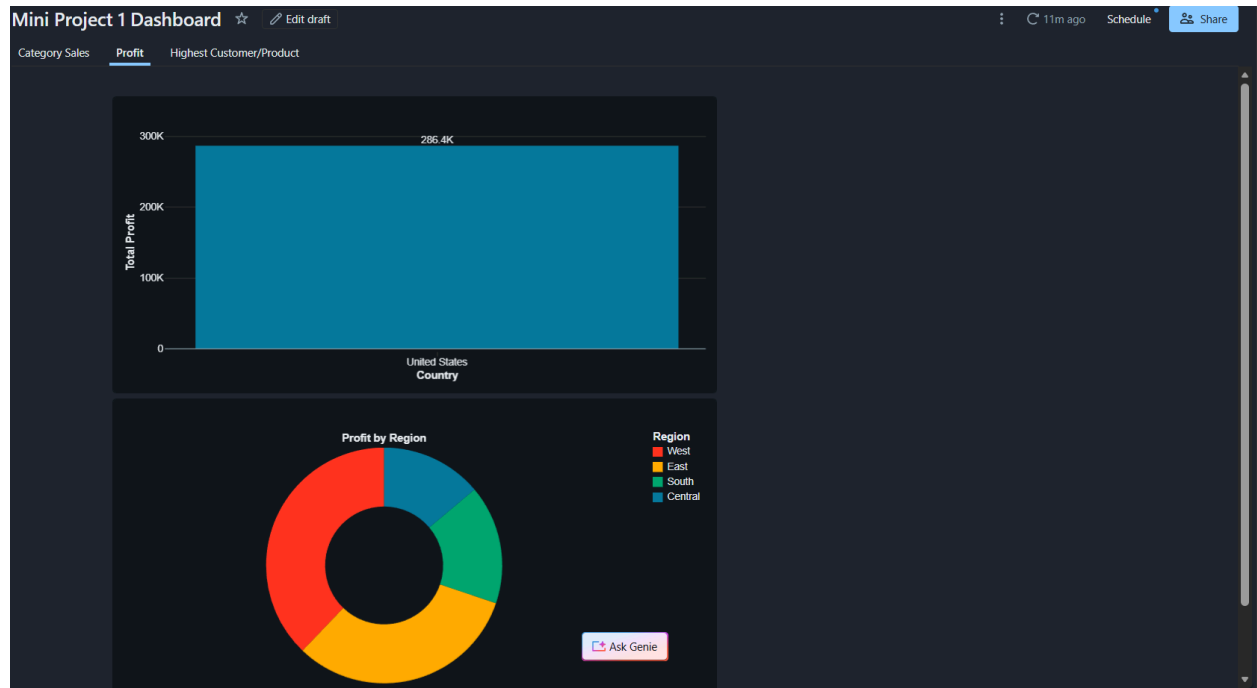
top_customers_by_sales_city.write.mode("overwrite").saveAsTable("gold_top_customers_by_sales_city")
> See performance (2) ✓ Checked for improvements
top_customers_by_sales_city: pyspark.sql.connect.dataframe.DataFrame = [Customer_Name: string, City: string ... 1 more field]

Table +
Customer_Name City 1.2 total_sales
1 Sean Miller Jacksonville 23861.228
2 Tamara Chand Lafayette 18336.73999999999...
3 Raymond Buch Seattle 14052.48
4 Tom Ashbrook New York City 13723.498000000000...
5 Becky Martin San Antonio 10539.896
6 Hunter Lopez Newark 10499.97
7 Sanjit Chand Minneapolis 9900.19
8 Adrian Barton Detroit 9892.74
9 Bill Shonely Lakewood 9135.189999999999
10 Sanjit Engle Arlington 8805.04

10 rows 4.285s runtime Refreshed 40m ago
```

Databricks Python Code





Mini Project 1 Dashboard Results

Databricks Mini Project 2 Document

▶ Just now (<1s) 1 Python

from pyspark.sql import functions as F

▶ Just now (<1s) 2

dbutils.widgets.dropdown("report_type", "Daily", ["Daily", "Weekly", "Monthly"])
report_type = dbutils.widgets.get("report_type")
print(f"Selected report type: {report_type}")

Selected report type: Daily

▶ Just now (2s) 3

file_path = "/Volumes/mini_projects/mp2/store/sales 2.csv"

raw_df = (
 spark.read
 .option("header", "true")
 .option("inferSchema", "true")
 .csv(file_path)
)

df = raw_df.withColumn("date", F.to_date("date"))

df.printSchema()
display(df.limit(5))

> [See performance \(1\)](#) ✓ Checked for improvements

> 2 dataframes: df, raw_df

▶ Just now (2s) 4 Python

df.write.mode("overwrite").saveAsTable("silver_sales")
df = spark.table("silver_sales")

> [See performance \(1\)](#) ✓ Checked for improvements

> df: pyspark.sql.connect.dataframe.DataFrame = [id: integer, date: date ... 2 more fields]

▶ 1 minute ago (1s) 5 Python

today = F.current_date()

if report_type == "Daily":
 start_date, end_date = (
 df.select(F.date_sub(today, 1).alias("d")).first()["d"],
 df.select(F.date_sub(today, 1).alias("d")).first()["d"],
)
elif report_type == "Weekly":
 start_date, end_date = (
 df.select(F.date_sub(today, 7).alias("d")).first()["d"],
 df.select(F.date_sub(today, 1).alias("d")).first()["d"],
)
elif report_type == "Monthly":
 start_date, end_date = (
 df.select(F.trunc(today, "MM").alias("d")).first()["d"],
 df.select(F.date_sub(today, 1).alias("d")).first()["d"],
)
else:
 raise ValueError(f"Unsupported report_type: {report_type}")

print(f"Report type : {report_type}")
print(f>Date window : {start_date} to {end_date}")

> [See performance \(2\)](#) ✓ Checked for improvements

Report type : Daily
Date window : 2026-09-11 to 2026-09-11

```
report_df = (
    df.filter((F.col("date") >= F.lit(start_date)) & (F.col("date") <= F.lit(end_date)))
    .groupBy("city")
    .agg(
        F.countDistinct("id").alias("total_orders"),
        F.sum("sale").alias("total_sales"),
    )
    .orderBy(F.desc("total_sales"))
)
```

report_df: pyspark.sql.connect.dataframe.DataFrame = [city: string, total_orders: long ... 1 more field]

```
output_path = f"/Volumes/mini_projects/mp2/store/report/sales_report_{report_type.lower()}"

(
    report_df.coalesce(1)
    .write.mode("overwrite")
    .option("header", "true")
    .csv(output_path)
)

print(f"CSV dump written to: {output_path}")
```

See performance (1) ✓ Checked for improvements

CSV dump written to: /Volumes/mini_projects/mp2/store/report/sales_report_daily

Databricks Python Code for Mini Project 2

store

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/mini_projects/mp2/store / report Refresh Create directory

Filter files and directories at this level

Name	Size	Last modified
No content in volume		

report starts with nothing in folder

```
report_type
Daily

Just now (1s) 7
output_path = f"/Volumes/mini_projects/mp2/store/report/sales_report_{report_type.lower()}"

(
  report_df.coalesce(1)
  .write.mode("overwrite")
  .option("header", "true")
  .csv(output_path)
)

print(f"CSV dump written to: {output_path}")
> See performance \(1\) ✓ Checked for improvements
CSV dump written to: /Volumes/mini_projects/mp2/store/report/sales_report_daily
```

First daily run performed.

store Overview Files Details Permissions Policies

Description

[AI generate](#) [Add](#)

[/Volumes/mini_projects/mp2/store / report / sales_report_daily](#) [Refresh](#) [Create directory](#)

Filter files and directories at this level

Name	Size	Last modified
_SUCCESS	0.00 B	30 seconds ago
_committed_550591557827066034	112.00 B	30 seconds ago
_started_550591557827066034	0.00 B	30 seconds ago
☐ part-00000-tid-550591557827066034-07b43679-f2e5-4735-8d6d-30...	30.00 B	30 seconds ago

part-00000-tid-550591557827066034-07b43679-f2e5-4735-8d6d-305c4cd3aecb-169-1-c000.csv

Folder with CSV file created.

```
report_type
Weekly

Just now (1s) 7
output_path = f"/Volumes/mini_projects/mp2/store/report/sales_report_{report_type.lower()}"

(
  report_df.coalesce(1)
  .write.mode("overwrite")
  .option("header", "true")
  .csv(output_path)
)

print(f"CSV dump written to: {output_path}")
CSV dump written to: /Volumes/mini_projects/mp2/store/report/sales_report_weekly
```

Changing to weekly re-runs and does a weekly report.

```
report_type
Monthly

Just now (2s) 7
output_path = f"/Volumes/mini_projects/mp2/store/report/sales_report_{report_type.lower()}"

(
    report_df.coalesce(1)
    .write.mode("overwrite")
    .option("header", "true")
    .csv(output_path)
)

print(f"CSV dump written to: {output_path}")

CSV dump written to: /Volumes/mini_projects/mp2/store/report/sales_report_monthly
```

Same with monthly.

store

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/mini_projects/mp2/store / report Refresh Create directory

Filter files and directories at this level

Name	Size	Last modified
sales_report_daily		
sales_report_monthly		
sales_report_weekly		

They have now all been created.

store

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/mini_projects/mp2/store / report / sales_report_weekly Refresh Create directory

Filter files and directories at this level

Name	Size	Last modified
_SUCCESS	0.00 B	1 minute ago
_committed_8966964805374392832	113.00 B	1 minute ago
_started_8966964805374392832	0.00 B	1 minute ago
part-00000-tid-8966964805374392832-c093173d-eec6-4421-88b5-b...	30.00 B	1 minute ago

part-00000-tid-8966964805374392832-c093173d-eec6-4421-88b5-b622a8c0dc00-176-1-c000.csv

store

OverviewFilesDetailsPermissionsPolicies

Description

AI generate

Add

/Volumes/mini_projects/mp2/store / report / sales_report_monthly

Refresh

Create directory

Filter files and directories at this level

Name	Size	Last modified
_SUCCESS	0.00 B	1 minute ago
_committed_1956640447891946462	113.00 B	1 minute ago
_started_1956640447891946462	0.00 B	1 minute ago
part-00000-tid-1956640447891946462-2fa96fcb-773b-4c21-a086-ed...	30.00 B	1 minute ago

part-00000-tid-1956640447891946462-2fa96fcb-773b-4c21-a086-ed064acff207-183-1-c000.csv